Deliverable 3 - INF201

G6_1: Zuzanna Magdalena Smolinska and Sondre Aasheim

Submission deadline: 13th Nov 2026

Hand-in information:

- This is a group submission, so one person submits on behalf of the pair/group
- Submit Python scripts (.py), a Jupyter notebook (ipynb) and a pdf of the same notebook
  - See Overview and objectives for a list of what your notebook should contain
- Include a link to your GitHub repository and a log of the commits
  - To generate the log, paste this into the terminal after your last commit: `git log --graph --pretty=format:"%h - %an, %ar : %s" > git_log.txt`
  - The repository should contain a `requirements.txt` with information on your virtual environment
  - Note: Remember to create a new virtual environment for each deliverable
- Include a discussion of GenAI use, for example a summary of the chat log as you have done earlier
  - This can be a separate file or be included in the Jupyter notebook
- Make a short (5 mins) video discussing your code and results
  - Note that you both have to contribute to the video
  - Either by recording it together
  - Or recording parts separately and editing it together
- We would appriciate it if use English for this assignment, both in the writing and in the video

Overview and objectives

In this assignment you will continue working on the Character class in `game_simulator_v1.py` that we created in lecture_08 (check out the Week 44 module), and expand this to create a mini-RPG in the form of a battle simulator. To do this you will update and extend the existing class and methods, introduce new classes allowing us to use dice and weapons, and create a combat manager class.

The rules for the game is as follows:

- The game you build should allow two characters to battle until one of them wins.
- The characters can use weapons to attack, where the damage dealt is determined by a dice roll.
- The characters can have different specialisations (warrior or mage).
- A character's HP (hit points, or health) must be between 0 and the max given for this specific character.
- The character dies when it reaches 0 HP.

Learning Objectives:

- Enforce data encapsulation using private attributes and properties.
- Implement custom classes to model discrete concepts (`Dice`, `Weapon`, `Battle`).¨
- Create subclasses inheriting from the `Character` class.
- Apply object composition by equipping characters with weapons and dice.

To organise your project, we recommend the following struckture:

- Separate modules (Python scripts) for each of the classes created
- A Jupyter Notebook containing the following:
  - A presentation of each class created (showing attributes and methods objects of the class has)
  - Simulated battles between different types of characters, with and without weapons

0.1: Imports

```
import random                                   # used by the Dice class to
roll random numbers
import sys                                      # used to check the Python
version
from pathlib import Path                        # used to check that our
module files exist
```

0.2: Environment check

```
# verifying which Python version the notebook is using
print("Python version:", sys.version.split()[0])

# checking that all our module files are in the same folder as the
notebook
for file_name in ["dice.py", "items.py", "character.py", "battle.py"]:
    print(f"{file_name} found:", Path(file_name).exists())
```

```
Python version: 3.14.6
dice.py found: True
items.py found: True
character.py found: True
battle.py found: True
```

Task 1: Update the Character class

When we created the Character class in the lecture we first included an `attack()` method, and afterwards indicated that we wanted hit points to be a protected feature `_hp`. This lead to the `Character.attack()` method changing the target's protected attribute. Modifying another object's private attribute is bad form! Your first task is to fix this.

- Implement a `take_damage()` method to be used when the `Character` object is injured. This method should only affect the characters *own* HP.

- Implement a `heal()` method to be used when the `Character` heals (in other words increase their hit points)
- Change the existing `attack()` method so that calls the target's `take_damage()` method rather than directly modifying the target's `_hp`.
- Add an `is_alive` property to the `Character` class. This should be read-only, and return True if `_hp`$ > 0$ and False otherwise.

1.1: `take_damage()` method

```
def take_damage(self, damage: int) -> int:
    """
    This method only changes the character's own HP.
    When another character attacks, it calls this method instead of
changing the HP directly.

    damage: how much HP the character loses (negative damage counts as
0)
    returns: the character's HP after the hit
    """

    damage = max(0, damage)                  # damage cannot be less
than 0

    self._hp -= damage                       # only change the
character's own HP
    self._hp = max(0, self._hp)              # HP cannot be less than 0


    if self._hp > 0:
        return self._hp                      # character is still
alive, return the new hp
    else:
        print(self.name, "has died")         # HP reached 0, so the
character has died
        return self._hp
```

1.2: `heal()` method

```
def heal(self, amount: int) -> int:
    """
    This method only changes the character's own HP.
    It increases the HP, but never above the character's max HP.

    amount: how much HP the character gains (negative healing counts
as 0)
    returns: the character's HP after healing
    """

    amount = max(0, amount)                      # healing cannot be
less than 0
```

```
    self._hp += amount                              # only change the
character's own HP
    self._hp = min(self._hp, self.hp_max)           # HP cannot go above
max HP

    return self._hp
```

1.3: `attack()` method

```
def attack(self, target: "Character") -> None:          # using quotes
as Character is not fully defined yet
    """
    This method is called when the character attacks another
character.
    The target takes care of its own HP by calling its take_damage()
method.

    target: the character being attacked
    """

    target.take_damage(self.damage)                     # let the
target handle its own damage
```

1.4: `is_alive` property

```
@property
def is_alive(self) -> bool:
    """
    Returns True if the character is alive (HP > 0), False otherwise.
    """

    return self._hp > 0
```

1.5: Code verification

```
# importing updated Character class from character.py
from character import Character

# creating two characters
fiona = Character(name="Fiona", hp_max=100, damage=10)
jolan = Character(name="Jolan", hp_max=50, damage=20)

# action: Jolan attacks Fiona
jolan.attack(fiona)
print(fiona)                                        # should be 80 HP

# action: Fiona heals 15 HP
fiona.heal(15)
```

```
print(fiona)                                        # should be 95 HP

# action: Fiona heals 50 HP, but cannot go over max
fiona.heal(50)
print(fiona)                                        # should be 100 HP
(not 145)

# action: Fiona attacks Jolan more times than needed to kill him
for _ in range(7):
    fiona.attack(jolan)

print(jolan)                                        # should be 0 HP (not
negative)
print("Is Jolan alive?", jolan.is_alive)            # should be False

# checking if is_alive property is read-only
try:
    jolan.is_alive = True                           # this should fail
except AttributeError as error:
    print("AttributeError:", error)                 # the error is
expected, so we print it instead
```

```
Character: Fiona with 80 HP and 10 damage
Character: Fiona with 95 HP and 10 damage
Character: Fiona with 100 HP and 10 damage
Jolan has died
Jolan has died
Jolan has died
Character: Jolan with 0 HP and 20 damage
Is Jolan alive? False
AttributeError: property 'is_alive' of 'Character' object has no
setter
```

Task 2: Introducing randomness with a Dice class

Currently, characters deal fixed damage. To introduce variable damage, create a separate `Dice` class.

- Create a `Dice` class that can create any n-sided Dice (n>=2), where the default number of sides is 6.
- Add a `roll()` method to roll the dice, using `random.randint(1, self.sides)`.
- Add a `__repr__` method that returns string representations like `Dice(sides=6)`.
- Implement a `__call__` method that allow you to roll the dice created by `d20 = Dice(sides=20)` by calling `d20.roll()`.
  - See the Python documentation of `__call__` or this example

2.1: Creation of the Dice class

```
class Dice:
    """
    Represents a dice used for random damage rolls.
    """

    def __init__(self, sides: int = 6) -> None:
        """
        Creating a dice with the given number of sides.
        Our dice needs at least 2 sides, otherwise stop with a clear
error.

        sides: how many sides the dice has (default is 6, must be at
least 2)
        """

        if sides < 2:
            raise ValueError("A dice must have at least 2 sides")

        # store the number of sides, so other methods can use it
        self.sides = sides

    def roll(self) -> int:
        """
        Rolling the dice and returning a random number from 1 to the
number of sides.
        """

        # randint includes values from both ends, so a 6-sided dice
gives 1, 2, 3, 4, 5 or 6
        return random.randint(1, self.sides)

    def __repr__(self) -> str:
        """
        Showing the dice as text, for example: Dice(sides=6).
        """

        return f"Dice(sides={self.sides})"

    def __call__(self) -> int:
        """
        Rolling the dice by calling it directly, for example: d20()
instead of d20.roll().
        """

        # calling the dice does the same thing as rolling it
        return self.roll()
```

2.2: Code verification

```
# importing the Dice class from dice.py
from dice import Dice

# creating a default dice and a 20-sided dice
d6 = Dice()                                                    # default
dice, 6 sides
d20 = Dice(sides=20)                                           # 20-sided
dice

# __repr__: showing the dice
print(d6, d20)                                                 # should
print: Dice(sides=6) Dice(sides=20)

# roll(): rolling each dice 10 times
print("d6 rolls: ", [d6.roll() for _ in range(10)])            # numbers
between 1 and 6
print("d20 rolls:", [d20.roll() for _ in range(10)])           # numbers
between 1 and 20

# __call__: rolling by calling the dice directly
print("d20() gives:", d20())                                   # a number
between 1 and 20

# checking that a dice with fewer than 2 sides is not allowed
try:
    Dice(sides=1)                                              # this
should fail
except ValueError as error:
    print("ValueError:", error)                                # the
error is expected, so we print it
```

```
Dice(sides=6) Dice(sides=20)
d6 rolls:  [2, 1, 2, 5, 1, 6, 5, 3, 1, 1]
d20 rolls: [13, 5, 18, 13, 10, 11, 13, 4, 18, 19]
d20() gives: 12
ValueError: A dice must have at least 2 sides
```

Task 3: Create items

Create a `Weapon` class in `items.py` that utilizes your `Dice` class for damage calculations. We call the module `items.py` rather than `weapons.py` so that it can be expanded to include other items your characters can hold.

- Define the class `Weapon` with the attributes `name` and `damage_dice: Dice`.
- Include a `roll_damage()` method that calls `self.damage_dice.roll()`
- Update `Character.__init__` to accept an optional weapon.
- If a character has an equipped weapon, `attack()` should roll damage from the weapon instead of using default base damage.

3.1: Creation of the Weapon class

```
class Weapon:
    """
    Represents a weapon that deals damage using a dice.
    """

    def __init__(self, name: str, damage_dice: Dice) -> None:
        """
        Creating a weapon with a name and a dice for damage.

        name: the name of the weapon
        damage_dice: the dice that is rolled to decide the damage
        """

        self.name = name
        self.damage_dice = damage_dice              # the weapon holds
its own dice

    def roll_damage(self) -> int:
        """
        Rolling the weapon's dice to decide how much damage it deals.
        """

        return self.damage_dice.roll()              # the dice does
the rolling

    def __repr__(self) -> str:
        """
        Showing the weapon as text.
        An example: Weapon(name='sword', damage_dice=Dice(sides=8)).
        """

        return f"Weapon(name={self.name!r},
damage_dice={self.damage_dice!r})"
```

3.2: Code verification

```
# importing the Weapon class from items.py
from items import Weapon

# creating two weapons, each with its own dice
sword = Weapon(name="Sword", damage_dice=Dice(sides=8))
axe = Weapon(name="Axe", damage_dice=Dice(sides=10))

print(sword)
print(axe)

# roll_damage(): rolling each weapon's damage 10 times
print("\nSword damage:", [sword.roll_damage() for _ in range(10)])   #
numbers between 1 and 8
```

```
print("Axe damage:  ", [axe.roll_damage() for _ in range(10)])     # numbers between 1 and 10

Weapon(name='Sword', damage_dice=Dice(sides=8))
Weapon(name='Axe', damage_dice=Dice(sides=10))

Sword damage: [2, 2, 5, 5, 2, 5, 3, 5, 7, 5]
Axe damage:   [6, 4, 9, 5, 6, 9, 8, 1, 9, 10]
```

3.3: Updating `Character.__init__`

```
def __init__(self, name: str, hp_max: int = 100, damage: int = 10, weapon: Weapon | None = None) -> None:
    """
    Creating a character with a name, maximum HP, base damage, AND an optional weapon.

    name: the name of the character
    hp_max: the maximum HP (default is 100)
    damage: base damage when attacking without a weapon (default is 10)
    weapon: an optional weapon (default is None, meaning no weapon)
    """

    self.name = name
    self._hp = hp_max                    # the character starts with full HP
    self.hp_max = hp_max
    self.damage = damage
    self.weapon = weapon
```

3.4: Attacking with a weapon

```
def attack(self, target: "Character") -> None:          # using quotes as Character is not fully defined yet
    """
    This method is called when the character attacks another character.
    The damage comes from the weapon if the character has one, otherwise from the base damage.
    """

    if self.weapon is not None:
        damage = self.weapon.roll_damage()              # with a weapon: roll the weapon's dice
    else:
        damage = self.damage                            # without a weapon: use base damage
```

```
    target.take_damage(damage)                              # let the
target handle its own damage
```

3.5: Code verification

```
from character import Character
from dice import Dice
from items import Weapon

# creating a weapon
axe = Weapon(name="Axe", damage_dice=Dice(sides=10))

# creating one character with a weapon and one without
fiona = Character(name="Fiona", hp_max=100, damage=10, weapon=axe)
jolan = Character(name="Jolan", hp_max=100, damage=20)

# Fiona attacks with her axe: damage should be random (1–10 each time)
for _ in range(3):
    fiona.attack(jolan)
    print(jolan)

# Jolan attacks without a weapon: damage should always be 20
for _ in range(3):
    jolan.attack(fiona)
    print(fiona)
```

```
Character: Jolan with 92 HP and 20 damage
Character: Jolan with 91 HP and 20 damage
Character: Jolan with 89 HP and 20 damage
Character: Fiona with 80 HP and 10 damage
Character: Fiona with 60 HP and 10 damage
Character: Fiona with 40 HP and 10 damage
```

Task 4: Character specialization

Extend the character module to include two specialised subclasses of `Character`.

1. `Warrior` subclass (defensive specialist)
   - New attribute: `armor: int`, with a default value of 5
   - Override the `Character.take_damage()` method by reducing the incoming damage by armor before applying it.
2. `Mage` subclass (resource-based attacker)
   - New attributes: `mana: int` and `mana_max: int`, both with default value 30
   - Magic attacks consume mana, so the mana available determines whether the mage can use magic.
   - Override the `Character.attack()` method:
     - If the mage has at least 10 mana left, spend 10 mana to deal a magical spell attack dealing double damage. This magic attack does not use any weapons, even if the character has one equipped.

- If the mage has less than 10 mana left, fall back to the base attack, and restore 5 mana. This base attack should use whichever weapon the character has equipped.

4.1: Creation of the Warrior class

```
class Warrior(Character):
    """
    A defensive character whose armor reduces incoming damage.
    """

    def __init__(self, name: str, hp_max: int = 100, damage: int = 10,
weapon: Weapon | None = None, armor: int = 5) -> None:
        """
        Creating a warrior character.

        armor: how much of each attack is blocked (default is 5)
        (the other parameters are the same as for Character)
        """

        super().__init__(name, hp_max, damage, weapon)
        self.armor = armor

    def take_damage(self, damage: int) -> int:
        """
        Reducing the incoming damage by the armor before it is taken
from the HP.

        damage: how much damage the attack deals before armor
        returns: the warrior's HP after the hit
        """

        reduced_damage = max(0, damage - self.armor)        # armor
cannot turn damage into healing
        return super().take_damage(reduced_damage)          #
Character.take_damage handles the HP rules
```

4.2: Code verification

```
# importing the Character, Warrior and Mage classes from character.py
from character import Character, Warrior, Mage

fiona = Warrior(name="Fiona", hp_max=100, damage=10)            #
armor is 5 by default, no weapon
jolan = Character(name="Jolan", hp_max=100, damage=20)          #
normal character, no weapon

print("Fiona's armor:", fiona.armor)                            #
should be 5

jolan.attack(fiona)                                             #
```

```
Jolan attacks with 20 damage: armor blocks 5, so Fiona should lose 15
HP
print(fiona)                                                          #
should be 85 HP

jolan.damage = 3                                                      #
Jolan is now weaker than Fiona's armor: the attack should be fully
blocked
jolan.attack(fiona)
print(fiona)                                                          #
should still be 85 HP

fiona.heal(10)                                                        # the
warrior should still have all Character methods, for example heal()
print(fiona)                                                          #
should be 95 HP
```

```
Fiona's armor: 5
Character: Fiona with 85 HP and 10 damage
Character: Fiona with 85 HP and 10 damage
Character: Fiona with 95 HP and 10 damage
```

4.3: Creation of the Mage class

A mage attacks with magic, which costs mana.

- With at least 10 mana, the mage casts a spell: it costs 10 mana and deals double the base damage. The spell does not use a weapon.
- With less than 10 mana, the mage does a normal attack (using its weapon, if it has one) and gets 5 mana back.

```
class Mage(Character):
    """
    A magic character whose spells cost mana.
    """

    def __init__(self, name: str, hp_max: int = 100, damage: int = 10,
weapon: Weapon | None = None, mana: int = 30, mana_max: int = 30) ->
None:
        """
        Creating a mage, with or without a weapon.

        mana: how much mana the mage starts with (default is 30)
        mana_max: the most mana the mage can have (default is 30)
        """

        super().__init__(name, hp_max, damage, weapon)
        self.mana_max = mana_max
        self.mana = min(mana, mana_max)                               # mana
cannot start above the max
```

```
    def attack(self, target: "Character") -> None:
        """
        Attacking with a spell if there is enough mana, otherwise with
a normal attack.
        """

        if self.mana >= 10:                                          #
checking if the mage has at least 10 mana to cast a spell
            # enough mana: cast a spell that deals double base damage
(no weapon)

            self.mana -= 10                                          #
losing 10 mana for the spell
            print(f"{self.name} casts a spell! ({self.mana} mana
left)")
            target.take_damage(self.damage * 2)                      # base
damage is doubled for the spell, and the target handles its own HP
        else:
            # not enough mana: normal attack (with weapon if there is
one), and regain 5 mana

            super().attack(target)
            self.mana = min(self.mana + 5, self.mana_max)        # mana
cannot go above the max
```

4.4: Code verification

```
# importing the Mage class from character.py
from character import Mage

# creating a mage with an axe, and a normal character with a lot of HP
(so she survives the test)
axe = Weapon(name="Axe", damage_dice=Dice(sides=10))

jolan = Mage(name="Jolan", hp_max=100, damage=10, weapon=axe)
fiona = Character(name="Fiona", hp_max=200, damage=10)

# Jolan attacks 6 times, we print Fiona's HP and Jolan's mana after
each attack

for attack_number in range(1, 7):
    jolan.attack(fiona)
    print(f"Attack {attack_number}: Fiona has {fiona.hp} HP, Jolan has
{jolan.mana} mana\n")
```

```
Jolan casts a spell! (20 mana left)
Attack 1: Fiona has 180 HP, Jolan has 20 mana
```

```
Jolan casts a spell! (10 mana left)
Attack 2: Fiona has 160 HP, Jolan has 10 mana

Jolan casts a spell! (0 mana left)
Attack 3: Fiona has 140 HP, Jolan has 0 mana

Attack 4: Fiona has 137 HP, Jolan has 5 mana

Attack 5: Fiona has 135 HP, Jolan has 10 mana

Jolan casts a spell! (0 mana left)
Attack 6: Fiona has 115 HP, Jolan has 0 mana
```

Task 5: Running battles

Create a `Battle` class to manage turn-based combat between two `Character` objects.

- Initialize `Battle` with `hero: Character` and `enemy: Character`.
- Implement a `play_round()` method where `hero` attacks `enemy`, and if `enemy` is still alive, `enemy` attacks `hero`.
- Implement a `simulate_battle()` method with a `while` loop that runs rounds until one character is no longer alive.

5.1: Creation of the Battle class

```python
class Battle:
    """
    Manages a turn-based fight between two characters.
    """

    def __init__(self, hero: Character, enemy: Character) -> None:
        """
        Setting up a battle between two characters.

        hero: the character who attacks first in each round
        enemy: the character who attacks second
        """

        self.hero = hero
        self.enemy = enemy

        self.round = 0                                  # how many rounds
have been played


    def play_round(self) -> None:
        """
        Playing the round: the hero attacks first, then the enemy
attacks back if still alive.
```

```
        """

        self.round += 1                                # a new round
starts
        print(f"--- Round {self.round} ---")

        self.hero.attack(self.enemy)
        print(f"{self.hero.name} attacks! {self.enemy.name} has
{self.enemy.hp} HP left")

        # the enemy can only attack back if it survived

        if self.enemy.is_alive:
            self.enemy.attack(self.hero)
            print(f"{self.enemy.name} attacks! {self.hero.name} has
{self.hero.hp} HP left")


    def simulate_battle(self, max_rounds: int = 100) -> Character |
None:
        """
        Playing rounds until one of the characters dies.

        max_rounds: safety limit, so the battle cannot go on forever
(default is 100)
        returns: the winner, or None if nobody won within the round
limit
        """

        # keep playing as long as both characters are alive

        while self.hero.is_alive and self.enemy.is_alive:
            if self.round >= max_rounds:
                print("The battle took too long, nobody wins!")
                return None
            self.play_round()

        # the loop has ended, so one character is dead: the other one
is the winner

        winner = self.hero if self.hero.is_alive else self.enemy
        print(f"{winner.name} wins after {self.round} rounds!")
        return winner
```

5.2: Code verification

```
# importing the Battle class from battle.py
from battle import Battle

axe = Weapon(name="Axe", damage_dice=Dice(sides=10))
```

```
fiona = Character(name="Fiona", hp_max=100, damage=10, weapon=axe)
jolan = Character(name="Jolan", hp_max=60, damage=8)                # no weapon

# setting up the battle and running it until someone wins
battle = Battle(hero=fiona, enemy=jolan)
winner = battle.simulate_battle()
```

```
--- Round 1 ---
Fiona attacks! Jolan has 55 HP left
Jolan attacks! Fiona has 92 HP left
--- Round 2 ---
Fiona attacks! Jolan has 52 HP left
Jolan attacks! Fiona has 84 HP left
--- Round 3 ---
Fiona attacks! Jolan has 43 HP left
Jolan attacks! Fiona has 76 HP left
--- Round 4 ---
Fiona attacks! Jolan has 35 HP left
Jolan attacks! Fiona has 68 HP left
--- Round 5 ---
Fiona attacks! Jolan has 34 HP left
Jolan attacks! Fiona has 60 HP left
--- Round 6 ---
Fiona attacks! Jolan has 25 HP left
Jolan attacks! Fiona has 52 HP left
--- Round 7 ---
Fiona attacks! Jolan has 20 HP left
Jolan attacks! Fiona has 44 HP left
--- Round 8 ---
Fiona attacks! Jolan has 13 HP left
Jolan attacks! Fiona has 36 HP left
--- Round 9 ---
Fiona attacks! Jolan has 12 HP left
Jolan attacks! Fiona has 28 HP left
--- Round 10 ---
Fiona attacks! Jolan has 4 HP left
Jolan attacks! Fiona has 20 HP left
--- Round 11 ---
Jolan has died
Fiona attacks! Jolan has 0 HP left
Fiona wins after 11 rounds!
```

5.3: Final battle simulation: Warrior vs Mage

```
# importing the Warrior and Mage classes from character.py
from character import Warrior, Mage

# creating a warrior with an axe and a mage without a weapon
```

```python
axe = Weapon(name="Axe", damage_dice=Dice(sides=10))

fiona = Warrior(name="Fiona", hp_max=100, damage=10, weapon=axe)
# armor is 5 by default
jolan = Mage(name="Jolan", hp_max=80, damage=10)
# 30 mana by default, no weapon

# running the battle until one of them wins
battle = Battle(hero=fiona, enemy=jolan)
winner = battle.simulate_battle()
```

```
--- Round 1 ---
Fiona attacks! Jolan has 70 HP left
Jolan casts a spell! (20 mana left)
Jolan attacks! Fiona has 85 HP left
--- Round 2 ---
Fiona attacks! Jolan has 68 HP left
Jolan casts a spell! (10 mana left)
Jolan attacks! Fiona has 70 HP left
--- Round 3 ---
Fiona attacks! Jolan has 67 HP left
Jolan casts a spell! (0 mana left)
Jolan attacks! Fiona has 55 HP left
--- Round 4 ---
Fiona attacks! Jolan has 57 HP left
Jolan attacks! Fiona has 50 HP left
--- Round 5 ---
Fiona attacks! Jolan has 50 HP left
Jolan attacks! Fiona has 45 HP left
--- Round 6 ---
Fiona attacks! Jolan has 47 HP left
Jolan casts a spell! (0 mana left)
Jolan attacks! Fiona has 30 HP left
--- Round 7 ---
Fiona attacks! Jolan has 45 HP left
Jolan attacks! Fiona has 25 HP left
--- Round 8 ---
Fiona attacks! Jolan has 40 HP left
Jolan attacks! Fiona has 20 HP left
--- Round 9 ---
Fiona attacks! Jolan has 36 HP left
Jolan casts a spell! (0 mana left)
Jolan attacks! Fiona has 5 HP left
--- Round 10 ---
Fiona attacks! Jolan has 35 HP left
Fiona has died
Jolan attacks! Fiona has 0 HP left
Jolan wins after 10 rounds!
```